Unity
オブジェクト別
デバッグログを作ろう！！

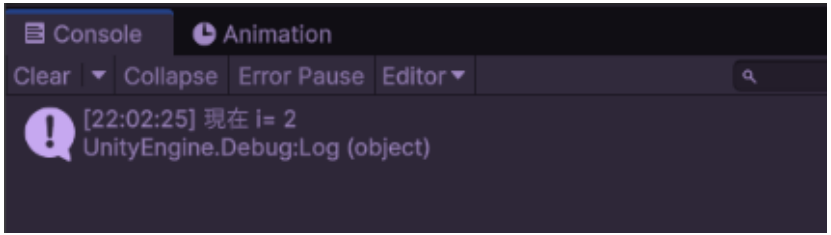
（AIにまかせて）

蒲焼さん太郎

Debug.Log(); 大好きですか？

```
int i = 2;
Debug.Log($"現在 i= {i}");
```





だいすきですよね！！？？

実行のタイミングで文字列を
表示するだけ

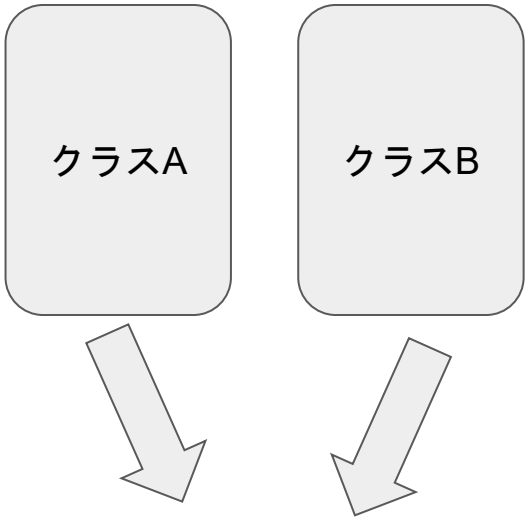
＝＞　現在の処理の場所

　　現在の変数の値

想像と違う出力を見つけたら、そこがバグ！！！！！

デバッグ神の神器！！！！！！！！！！

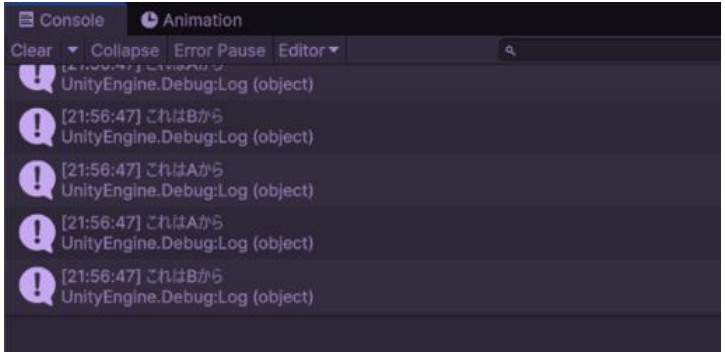
Debug.Log(); 君の難点



知りたいLogを見つけるのがめんどい

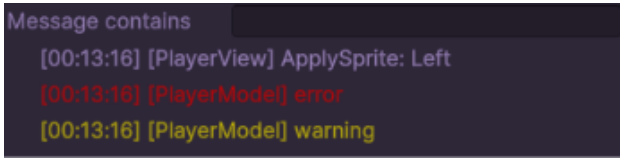
Q. いや、使ったLogコメントアウトしろよ

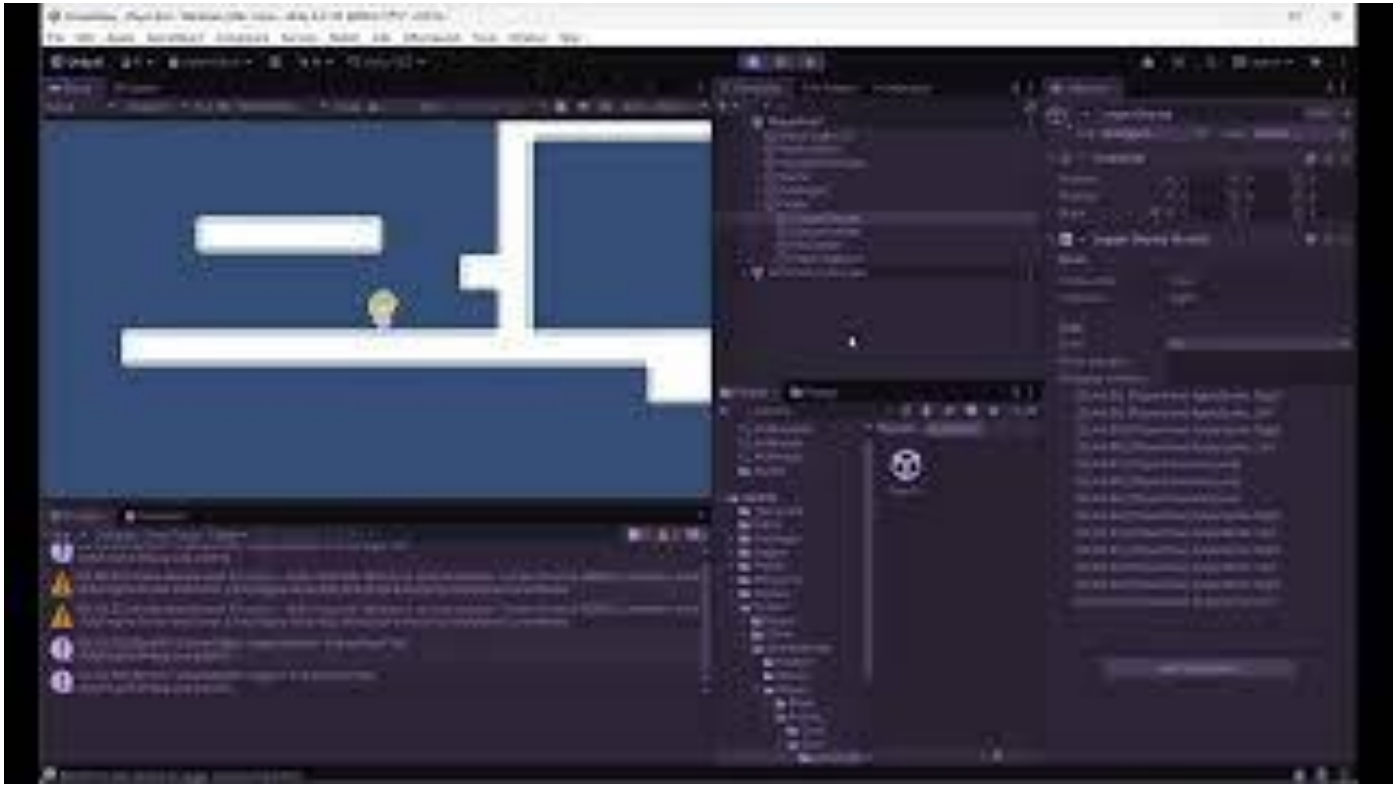
A. 忘れ,,,,じゃなくて、Logを残すメリットもある
- 再現性の低いバグ
  ＝＞Logを付けたい、けどまた初めから
  ＝＞バグが再現できない
- いつでも確認しておきたい変数



じゃあGameObjectごとに管理すればいいじゃん

作るぞ！！AIが

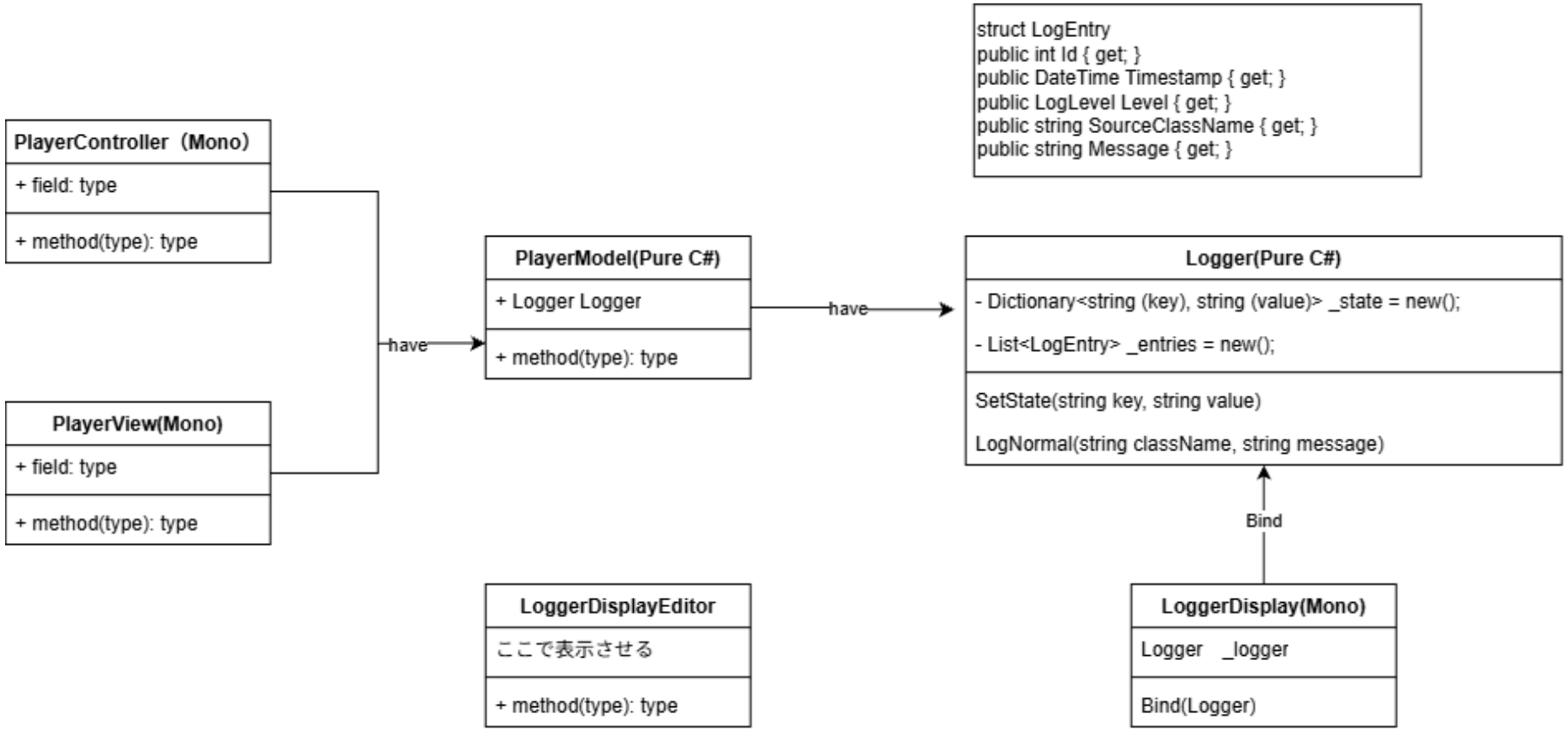




1. Log表示
- Level(Normal, Warning, Error)
- 時間
- クラス名
- 内容
- 検索機能

1. パラメータの監視

クラス図


struct LogEntry
public int Id { get; }
public DateTime Timestamp { get; }
public LogLevel Level { get; }
public string SourceClassName { get; }
public string Message { get; }
PlayerController（Mono）
+ field: type
+ method(type): type
PlayerModel(Pure C#)
+ Logger Logger
+ method(type): type
have
have
Logger(Pure C#)
- Dictionary<string (key), string (value)> _state = new();
- List<LogEntry> _entries = new();
SetState(string key, string value)
LogNormal(string className, string message)
PlayerView(Mono)
+ field: type
+ method(type): type
Bind
LoggerDisplayEditor
ここで表示させる
+ method(type): type
LoggerDisplay(Mono)
Logger _logger
Bind(Logger)

パラメータの監視方法について

U3のReactiveProperty<T>を使用　（値が変わったら任意の関数を呼べる）

```
Frequently called  6 usages
public ReactiveProperty<bool> IsGrounded { get; } = new(false);
public ReactiveProperty<Direction> Direction = new(Player.Direction.Left);

Frequently called  2 usages  kabayaki944 *
public PlayerModel()
{
    Watch("IsGrounded", IsGrounded);
    Watch("Direction", Direction);
}
```

```
public void Watch<T>(string key, Observable<T> property)
    => LoggerHelper.Watch(key, property, Disposables, Logger);
```

```
public static void Watch<T>(string key, Observable<T> property, ICollection<IDisposable> disposable,
    ILogger Logger)
{
    property
        .Subscribe(value => Logger.SetState(key, value?.ToString() ?? "null"))
        .AddTo(disposable);
}
```

良きデバッグライフを！！！